Claims triage: design note

Aditya Verma, Senior Data Scientist assignment, Apeiro Digital

Labels

The CMS synthetic data has no outcomes, so I injected five patterns into about 5% of claims and labelled them. The model never sees which claims were injected.

Pattern	What I changed	Label	Claims
Diagnosis substitution	Principal diagnosis replaced by one with a different ICD-10 first letter	Denied	8,858
Coverage ended	Service date falls in a simulated coverage lapse	Denied	3,596
Duplicate	Claim re-submitted 0 to 2 days later, received 3 to 30 days later	FWA	3,623
Overbilling	Billed amount multiplied by 2 to 6	FWA	3,221
Provider-day spike	4 to 12 extra claims copied onto one provider-day	FWA	2,459

Noise in both directions: 12% of injected claims are labelled clean, and 1.15% of clean claims are labelled denied or FWA for reasons not in the data. Final balance: 94.2% clean, 3.6% denied, 2.1% FWA.

Trust. The results measure recovery of injected patterns, not real-world fraud detection, and are not a production forecast. Each pattern is cleaner than reality: a substituted diagnosis is not verified as medically incompatible, and a multiplied amount ignores legitimate changes in treatment. Spike copies and replacement diagnoses are drawn from all periods, a generator limitation.

Feature design

- **Code grouping.** HCPCS grouped into sections; ICD-10 by first letter and three-character category. Raw codes are too sparse to learn from.
- **Diagnosis to procedure consistency.** The share of past claims for this procedure carrying this diagnosis category, and the share of the member's own claims in the past year with it.
- **Peer comparison.** Billed amount against the median for the same procedure combination, and against this provider's and this member's usual amount.
- **Time windows.** Member claims in 30 and 365 days, same procedure in 7 days, same-day and same-amount repeats, provider claims that day against its usual.
- **Coverage.** Days after the member's coverage end. This restates the label rule: a rule, not learned detection.

Leakage controls. Everything is keyed on the date a claim is received. The split is by received date: train 2015 to 2020, validation 2021, test 2022 to 2023. Reference tables only use claims received before the training cutoff. History features only use claims received on an earlier date, with one exception: provider-day volume also counts claims received the same date, because scoring is an end-of-day batch. No feature uses labels. Fixture tests and an audit that recomputes features with all later claims deleted both pass.

Member diagnosis history: kept, with a caveat. These four features carry most of the gain: retraining without them drops validation precision at capacity from 80% to 69% and recall from 76% to 66%. They are leak-free but flattered by the data. Among claims with at least three prior claims in the past year, the diagnosis category is new to the member on 3% of non-injected claims against 84% of injected substitutions, because synthetic members have unusually stable diagnoses and my replacements ignore member history. On real claims new diagnoses are often legitimate, so this gain may not transfer. Already here, 7% of non-injected claims with a new diagnosis are selected, against 0.4% otherwise.

Rejected as generator artifacts. Strict-timing provider-day volume and receipt lag: both only detect how my generator sets received dates.

Class imbalance

Chosen: LightGBM with no class weights or resampling; imbalance is handled at the decision step through cost-based thresholds. Unweighted probabilities track observed rates reasonably on validation (`outputs/calibration_validation.csv`).

Rejected: balanced class weights (ranking no better at PR-AUC 0.70 against 0.71, probabilities inflated); SMOTE (blending rows makes no sense for codes and counts); undersampling (throws away data and distorts probabilities).

Evaluation

Accuracy is meaningless at 94% clean, so I used PR-AUC and precision and recall at review capacity (exactly the top 6% of claims by score).

Model (test set)	PR-AUC bad	PR-AUC FWA	Precision at capacity	Recall at capacity
Logistic regression	0.50	0.55	57%	58%
Decision tree, depth 6	0.64	0.65	70%	71%
Random forest	0.70	0.74	75%	76%
LightGBM (final)	0.71	0.75	75%	76%

Random ranking gives about 0.06, and 18% of bad labels are pure noise, so roughly 82% of bad claims carry any signal. Random forest ties LightGBM, which I chose on validation; it is faster and gives exact per-claim contributions. Larger trees gave under one point and were not adopted. Choices were made on validation, but test numbers were printed during development, so the test set is not a pristine holdout.

Thresholds and cost. Illustrative assumptions: a standard review costs $15, a senior review $60, and paying a bad claim loses its whole billed amount (for overbilling the real loss is only the excess). A standard reviewer stops 90% of denials and 60% of FWA; a senior reviewer 95% of both. Capacity is 8% of claims for review and 2% for high risk, over the whole period, not per day. The cheapest pair in the searched grid on validation: review if P(bad) >= 0.06, high risk if P(FWA) >= 0.40. This routing point is separate from the 6% evaluation point.

Strategy (test set, 455 days, 79,380 claims)	Total cost	Reviews per day
Pay everything	$12.23M	0
Review everything (ignores capacity)	$4.16M	175
Model routing	$3.38M	14

Routing flags 7.8% of claims and 79% of bad claims; the high-risk queue is 83% bad and the review queue 52%.

Explainability

LightGBM gives each feature's contribution to a claim's score. Related features are grouped into one reason, shown only if it raised the score and a plain check holds. Example: "Billed amount $3,380 is 3.8 times the typical $887 for this procedure."

Where they could mislead:

- They describe why the model scored the claim, not proof of wrongdoing.
- Correlated features share credit, so the reason shown may not be the real cause.
- "New diagnosis for this member" is often legitimate: a prompt to check, not a finding.
- High provider-day volume is a signal about the day, not about each claim on it.
- 7% of flagged claims get only a generic sentence.

Deployment and monitoring

- **Batch or real time.** End-of-day batches, matching the features; the API rescores a claim against available history. Coverage and exact duplicates are candidates for rule-based checks.
- **Drift and code sets.** Track weekly feature distributions (PSI), queue sizes and the share of unseen codes. ICD-10 and HCPCS update yearly; reference tables are rebuilt.
- **Feedback.** Reviewer decisions become labels. Auto-approved claims never get reviewed, so audit a random sample with known sampling rates to correct for selective labelling.
- **Retraining.** Quarterly, or when precision at capacity drops or drift alerts fire.
- **Audit trail.** Per claim: input, features, model version, thresholds, score, queue, reasons, reviewer decision and timestamps.

With another week

- Rebuild the generator: clinically grounded mismatches, some drawn from member history, donors from earlier periods only, realistic received dates.
- Add unbundling, using NCCI procedure-pair edits, and standard groupers (CCS, BETOS).
- Route on expected benefit per queue (probability times amount, less review cost) with daily capacity.